HPPK with Nothing: A Universal All-Zero Forgery against HPPK DS

Martin Feussner
Selmer Center, University of Bergen
martin.feussner@uib.no

14 October 2026

This audit used the OpenAI model Daybreak Blue at the ultra reasoning level. OpenAI Codex generated and selected this paper's title and intentionally chose "HPPK with Nothing" as wordplay: HPPK visually echoes "happy," while the forgery consists entirely of zeros. OpenAI Codex independently reconstructed, derived, and verified the attack and performed AI-assisted attack-history research, algebraic cryptanalysis, implementation, experimentation, and technical writing. Separately tasked AI agents performed algebra/domain review, hostile adversarial review, and fresh-context independent reproduction, and a final exact-target history recheck. Primary reproduction, adversarial review, independent reproduction, final history review, and independent AI Publication Review passed. The result is classified as a verified public design attack. No independent human verification was performed; it is optional and is not a publication requirement.

Abstract

We describe a public-key-only universal forgery against the Homomorphic Polynomial Public Key Cryptography for Quantum-secure Digital Signature (HPPK DS) construction in IACR ePrint 2023/1768. For every public coefficient, the verifier computes two Barrett-style expressions: each contains an outer product with one signature component and a floor whose numerator contains that same component. Substituting $F = H = 0$ annihilates both outer products and sets both floor arguments to zero, so the final verification equation is $0 = 0$ for every message, structurally valid public key, and verifier-noise choice. The advertised SHA-256, SHA-384, and SHA-512 profiles split a complete hash into four field-sized representatives; the full mathematical forgery is therefore four copies of $(0, 0)$. It makes zero signing queries and uses no secret information.

The proof covers all three rows of the current target's Table 1 and all fifteen rows of the benchmark update's Table 2. A standard-library Python reproducer checks the complete four-pair form and live controls for all eighteen rows. A separate experiment against public `xtaci/hppk` commit `a66c951eef2f231a228e8121a7a6391b991146e1` accepted 45 of 45 attacker-created, JSON-round-tripped zero objects across 15 fresh keys through the ordinary exported verifier, with zero attack signing queries. That source uses a different digest boundary, signature format, and parameter generation; it is corroboration, not a row-exact implementation of the paper tables. A fresh-context agent independently recreated the paper and source tests. Its paper run passed 72/72 complete four-pair forgeries across all 18 rows, and its pinned-source run accepted 192/192 attacker checks across 12 fresh keys and 48 fresh messages while 96/96 honest checks accepted and 288/288 negative controls rejected. The attack used no signing queries or secret reads. Final exact-target history review and independent Publication Review passed.

1 Introduction

Kuang, Perepechaenko, Sayed, and Lou propose HPPK DS by extending polynomial public-key constructions over two hidden rings and transforming modular multiplication into a public verification relation using Barrett-style quotients [1]. The paper argues that the resulting floor terms make forgery nonlinear and assigns exponential work to generic searches over the signature values.

The boundary value zero bypasses that intended obstacle. Each transformed coefficient contains an outer product with F or H and a Barrett correction whose floor argument contains the same component.

Both components may be zero under the stated mathematical domain, and the verifier has no nonzero check. The attack therefore requires no search: for every hash segment, output the pair $(0, 0)$.

This observation is elementary. We make no claim that zero substitution is a new cryptanalytic technique. The contribution is the exact application to the ordinary HPPK DS verifier, its complete four-segment signature, every advertised row, and a separately scoped public-source implementation check.

Claim boundary. The paper-level claim is a universal forgery against the equations in the current ePrint target and the benchmark's clarified row set. The paper defines no canonical byte encoding. No public or reviewed row-exact implementation or archived vector was located for a table row. The public Go source is tested under its own source-specific format and parameters and is never presented as execution of a paper row.

2 Target, versions, and parameters

2.1 Pinned target

The target is *Homomorphic Polynomial Public Key Cryptography for Quantum-secure Digital Signature*, IACR ePrint 2023/1768 [1]. The pinned current PDF has SHA-256 digest

`faa261feaf9e1f030273d311bfe642504a82d9f0759524446f9c86afbbc269c1.`

The official record reports receipt on 15 November 2023 and revision on 17 November 2023; the body matches arXiv 2311.08967v2. Later arXiv v3 is useful lineage but is not silently substituted for the current ePrint target.

The performance and parameter update is *Benchmark Performance of Homomorphic Polynomial Public Key Cryptography for Key Encapsulation and Digital Signature Schemes*, IACR ePrint 2024/019 [2]. Its pinned PDF has SHA-256 digest

`1ee259abede8b393f26e021fccbd41774b212146717a63f646aee282273198a7.`

2.2 Advertised rows

The target Table 1 gives three security levels but omits a concrete prime and an explicit (n, λ, m) tuple. Its surrounding construction supports the contextual values $n = \lambda = 1, m = 2$; we do not turn that inference into an exact table cell. Table 1 records the values that are actually printed.

Level	field bits	hidden bits	Barrett bits	signature bytes	hash
I	64	128	160	128	SHA-256
III	96	192	224	192	SHA-384
V	128	256	288	256	SHA-512

Table 1: Current ePrint 2023/1768 Table 1 rows. A complete hash contains four field-sized segments in every row.

Benchmark Table 2 makes fifteen tuples explicit. All use $n = \lambda = 1$; Table 2 groups the five rows at each level. The package's `affected-rows.json` retains every row individually.

Level	p	(m, L, K) tuples	hash	signature bytes
I	$2^{64} - 59$	$(2, 144, 288)$, $(2, 144, 208)$, $(2, 144, 176)$, $(1, 144, 208)$, $(1, 144, 176)$	SHA-256	144
III	$2^{96} - 17$	$(2, 208, 416)$, $(2, 208, 272)$, $(2, 208, 240)$, $(1, 208, 272)$, $(1, 208, 240)$	SHA-384	208
V	$2^{128} - 159$	$(2, 272, 544)$, $(2, 272, 336)$, $(2, 272, 304)$, $(1, 272, 336)$, $(1, 272, 304)$	SHA-512	272

Table 2: All fifteen ePrint 2024/019 Table 2 rows.

2.3 Claimed security

The original HPPK DS paper sizes its parameter sets from the claimed $O(S_1S_2/p) = O(p^3)$ key-recovery cost and reports estimated entropies of 192, 288, and 384 bits in Table 1. Propositions 3–5 separately present generic forgery searches with claimed cost $O(S_1S_2)$. The companion benchmark later presents an optimized modulus-recovery search with claimed total cost $O(2^L)$; its Table 2 sets $L = 2|p|_2 + 16$ and labels Levels I, III, and V with 144, 208, and 272 bits. The all-zero forgery bypasses these searches: it constructs the fixed four-pair signature directly and does not depend on either claimed entropy column.

3 HPPK DS verification

Let p be the public field modulus. The scheme forms product polynomials $P = fB$ and $Q = hB$, encrypts their coefficients over hidden rings with moduli S_1, S_2, and publishes Barrett-transformed data. For coefficient indices $i \in \{0, \ldots, n + \lambda\}$ and $j \in \{1, \ldots, m\}$, write

$$p'_{ij} = \beta P_{ij} \bmod p, \qquad q'_{ij} = \beta Q_{ij} \bmod p,$$

$$s_1 = \beta S_1 \bmod p, \qquad s_2 = \beta S_2 \bmod p,$$

and let μ_{ij}, ν_{ij} be the published integer quotient approximations for a positive power of two R.

For one field-sized message representative x, signing samples $\alpha \in \mathbb{F}_p$ and produces residues F and H modulo the two hidden rings. The stated domain includes $\alpha = 0$, and no signing or verification rule excludes the resulting zero pair.

Target Equation (28) embeds the signature in the two coefficient equations

$$U_{ij}(H) = Hp'_{ij} - s_1 \left\lfloor \frac{H\mu_{ij}}{R} \right\rfloor \bmod p, \tag{1}$$

$$V_{ij}(F) = Fq'_{ij} - s_2 \left\lfloor \frac{F\nu_{ij}}{R} \right\rfloor \bmod p. \tag{2}$$

The target aggregates these coefficients with powers x^i and verifier noise values u_j, then accepts when the two aggregates are equal. The benchmark checks the corresponding equality separately for each j. Neither form tests that F or H is nonzero.

Hashes wider than the field are split into field-sized segments and each segment is signed independently. SHA-256 over a 64-bit field, SHA-384 over a 96-bit field, and SHA-512 over a 128-bit field each produce exactly four segments. Thus a full mathematical signature contains four pairs.

4 Universal all-zero forgery

Lemma 1 (Zero coefficients)**.** *For every structurally valid public coefficient, field representative, and positive Barrett power R, substituting $F = H = 0$ in Equations (1) and (2) gives $U_{ij} = V_{ij} = 0$.*

Proof. Both products outside the floor vanish. Inside the floor, $0\mu_{ij}/R = 0$ and $0\nu_{ij}/R = 0$, so both floor values are zero. Reduction modulo p preserves zero. This is exact and does not depend on the quality of the Barrett approximation. □

Theorem 1 (Complete-hash universal forgery)**.** *For each advertised HPPK DS row, the four-pair signature*

$$\big((0,0),(0,0),(0,0),(0,0)\big)$$

is accepted for every complete message hash, every structurally valid public key, and every verifier-noise choice by the ordinary published verifier.

Proof. Apply the lemma to every coefficient for one segment. Every polynomial sum is zero, so both the target's noise-weighted equation and every benchmark per-j equation compare zero with zero. Repeat independently for all four hash segments. All four comparisons accept. The conclusion is independent of the message representatives and public coefficients. □

The attacker chooses any fresh message, emits eight zero integers, and makes no signing query. Work and output are linear only in the fixed four-segment signature length. Under any natural fixed-width unsigned encoding, the object would be an all-zero byte string of the printed signature length. Because the papers do not define a canonical wire format, this observation is illustrative rather than an interoperability claim.

5 Paper-level executable reproduction

The public reproducer uses only the Python standard library. It transcribes all three target rows and all fifteen benchmark rows. For each row it hashes explicit messages with the named hash function, splits the digest into four field-sized representatives, constructs four zero pairs, evaluates every coefficient with integer floor division, and checks both the per-j and noise-aggregated equalities.

Target Table 1 does not publish a prime. For those three records the executable uses the corresponding later benchmark prime as a labeled arithmetic fixture. The result does not depend on that choice; it is not reported as a target-row parameter. The benchmark records use their exact Table 2 primes, dimensions, L, and K. The coefficients are deterministic audit fixtures, not author keys. Accordingly, the executable checks the exact universal identity while making no row-exact implementation claim.

The quick reference checks one complete hash per row, for 18 messages and 72 zero pairs. The full reference checks four complete hashes per row, for 72 messages and 288 zero pairs. Both pass every row with zero signing queries. Each row also requires:

- rejection of three-pair and five-pair objects;
- rejection when only F or only H is changed to one under a fixture whose constant coefficient makes the comparison path live;
- rejection of the zero object under a test-only per-segment repair; and
- acceptance of a balanced nonzero equation fixture under that repair.

The last item is a path control, not a claim that the fixture is produced by the unavailable row-exact signer.

6 Separate public Go implementation evidence

The public `xtaci/hppk` repository was pinned at commit [6]

`a66c951eef2f231a228e8121a7a6391b991146e1.`

The retained archive has SHA-256 digest

`f013a87fab81955e814d23e0ad869454523898b560285dc9664a1b474a61f28c.`

Its exact 19-file inventory validated before and after the experiment, and `hppk.go` remained SHA-256

`052291b33de3c5a6a64d5de1752da6c3af01b92cdbbdf3862545c60dee1dea91.`

This source is materially different from the papers. It consumes one entire digest as one integer, requires key order at least five, generates different hidden moduli and Barrett values, and places `Beta`, `S1Verify`, `S2Verify`, `U`, `V`, and `K` inside the signature object. No source configuration is an exact advertised row.

The primary harness nevertheless tests the same zero boundary through the ordinary exported `hppk.VerifySignature` function. For each trial it uses a fresh source-generated key, builds a complete attacker object from public array lengths and constants, serializes it to JSON, decodes it into a fresh signature, and verifies the forged object before any signing call. Five source-specific profiles were exercised:

- custom primes $2^{64} - 59$, $2^{96} - 17$, and $2^{128} - 159$, each at order five with SHA-256, SHA-384, and SHA-512 respectively; and
- the source's 2,049-bit `DefaultPrime` at orders five and ten.

Three fresh keys per profile and three distinct messages per key gave 15 unique public-key fingerprints and 45/45 accepted forgeries. The attack made zero signing queries. An object with nonzero auxiliary values but $F = H = 0$ also accepted for every key, isolating the two paper components as the cause.

After attack construction and verification, fifteen control-only calls to `Sign` established honest acceptance. Honest changed-message, $F+1$ mutation, targeted nonzero, malformed-length, nil-signature, empty-digest,

and wrong-key controls rejected as registered. These later signing calls are never counted as attack queries.

All generated keys had the intended shape, and ordinary honest signatures verified. The diagnostic `PublicKey.IsValid()` nevertheless returned false. Key generation maps coefficients into the hidden-ring range, whereas that diagnostic requires them to be below the public prime. The ordinary verifier does not call `IsValid`. This is a separate source consistency issue, not the attack mechanism.

The host's Go 1.22.2 rejected the source's Go 1.22.5 module directive. A prospectively registered fallback copied the verified source byte for byte to an offline temporary GOPATH, disabled module and network resolution, and built without source edits. Pre- and post-build recursive comparisons were empty. The complete registered workflow used 24.59 seconds wall time, 19.20 CPU seconds, 230,900 KiB peak resident memory, and one worker. The implementation harness itself used 0.04 seconds wall time, 0.04 CPU seconds, and 6,144 KiB peak resident memory.

6.1 Fresh-context independent reproduction

A separately tasked AI agent reconstructed the equation and source harnesses from the public target materials and pinned source. At the paper level, all 3 target and 15 benchmark rows passed: 72/72 complete fresh-message signatures, 288/288 segments, and 1,440 coefficient pairs. Each complete signature used four zero pairs. Signing queries and secret reads were zero.

The separate source run covered custom 64-, 96-, and 128-bit primes and the source `DefaultPrime` at orders 5, 8, and 10. It generated 12 unique fresh keys and 48 unique fresh messages. The ordinary exported verifier accepted both all-zero and live-auxiliary attacker objects under correct and wrong keys, for 192/192 attacker checks. The live objects retained $F = H = 0$ while using nonzero in-range auxiliary values, legitimate lengths, and the source-derived normal positive `K`. Honest direct and serialized checks accepted 96/96; six negative-control classes rejected 288/288. Source inventories matched exactly before and after. The source run used one worker, 0.34 seconds wall time, and 7,680 KiB peak resident memory.

Attempt 1 completed the paper checks, then stopped at compilation before a source binary was created or run because of a pointer-slice bookkeeping type error. A prospectively sealed one-line type correction enabled the sole source execution. No source-result tuning or retry occurred.

7 Specification inconsistencies and limitations

The result survives several specification ambiguities, but the public report must retain them:

1. **No paper wire format.** The papers provide size formulas but no canonical component ranges, endianness, integer encoding, or parser. The complete paper claim is therefore four mathematical pairs.
2. **No located row-exact executable.** No public or reviewed row-exact implementation or archived vector was located for any target or benchmark row. The proof is full-parameter because both coefficient expressions evaluate to zero independently of the parameter values, not because a table implementation was run.
3. **Broken benchmark pseudocode.** Algorithm 5 duplicates F and omits the evident H line. Algorithm 6 prints an extra coefficient index and uses s_1 where the equations require s_2. Nearby prose gives conflicting signature sizes, and a caption gives Level-I $L = 168$ against Table 2's $L = 144$. At $F = H = 0$, both printed floor arguments and both outer products are zero under either branch; the claim is anchored to target Equation (28), its signing and verification text, benchmark Equation (8), and Table 2.
4. **Source divergence.** The one-object Go verifier result is source-specific corroboration and cannot be used as a four-segment or row-exact execution claim.
5. **Attack type.** This is a universal fresh-message forgery, not secret-key or hidden-ring recovery.
6. **Publication gate.** Independent Publication Review passed.

8 Attack history and attribution

In public searches completed through 14 October 2026 at 14:47:51 UTC, we located no previous report of the four-pair all-zero universal forgery, or of another practical signing compromise, against the exact

Equation (28) HPPK DS paper construction in ePrint 2023/1768. This is a bounded, dated conclusion with moderate-high confidence. It cannot exclude nonpublic, deleted, unpublished, or unindexed disclosures, and search and citation indexes can lag. We claim neither absolute priority nor technique novelty.

The final search did locate a September 2026 public-key-only forgery against a materially divergent HPPK-labelled implementation, reported in a draft pull request and explicit public reproduction commit [4]. That implementation's attacker-supplied signature contains (F, H, U, V). Its test chooses $F = H = 1$, computes U and V from the message and public key at $p = 65537$, and passes a verifier that separately checks $U = \alpha H \bmod p$ and $V = \beta F \bmod p$. The paper target instead supplies only (F, H): its verifier derives every $U_{ij}(H)$ and $V_{ij}(F)$ internally and compares the two polynomial sides under Equation (28). The September finding is a real prior implementation-specific forgery, but it neither reports the four-pair all-zero attack nor executes an advertised paper row. The repository owner is not a target-paper author, and no target association was located. We classify it as related work against a divergent implementation, not an attack on the exact paper construction.

The target paper's Propositions 3–5 present generic forgery searches with claimed cost $O(S_1 S_2)$ and do not give a practical advertised-parameter compromise. Guo's practical cryptanalysis of MPPK/DS attacks a distinct predecessor with a different public structure and linear verifier [3]. The September 2026 D-HPPK KEM preprint reports a Lattice Reconstitution Attack against the original unmasked HPPK KEM [5]; it does not analyze the ePrint 2023/1768 signature verifier, recover a DS signing witness, or demonstrate fresh DS signing. These items are relevant history, but each falls outside the exact Equation (28) signature target.

9 Narrow repair direction

For every hash segment, verification should reject when both $F = 0$ and $H = 0$. The signer must resample α, or the resulting pair, whenever it would emit the newly forbidden value. These checks block the exact paper-level forgery in this report.

This is a narrow mitigation, not a proof of unforgeability. A complete revision should define canonical component ranges and encoding and should repeat its security and full-parameter analysis. The divergent Go format additionally needs coherent range/domain validation and binding of its auxiliary values; those questions are separate from the minimal paper repair.

10 Reproducibility and review status

The public artifact includes the full standard-library source, all 18 affected rows, deterministic quick and full reference outputs, package checks, build files, and this LaTeX source. Ordinary runs use operating-system entropy in their message labels; reference runs use a disclosed fixed fixture. Every run is sequential with one worker and emits machine-readable JSON ending in `ATTACK REPRODUCTION: PASS` on success.

Primary reproduction, hostile adversarial review, fresh-context independent reproduction, final exact-target history review, and independent Publication Review passed. The package is classified as a verified public design attack. Its final file inventory is sealed by the accompanying `SHA256SUMS` manifest.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

References

[1] Randy Kuang, Maria Perepechaenko, Mahmoud Sayed, and Dafu Lou. Homomorphic Polynomial Public Key Cryptography for Quantum-secure Digital Signature. IACR Cryptology ePrint Archive, Report 2023/1768, 2023. https://eprint.iacr.org/2023/1768.

[2] Randy Kuang, Maria Perepechaenko, Dafu Lou, and Brinda Tank. Benchmark Performance of Homomorphic Polynomial Public Key Cryptography for Key Encapsulation and Digital Signature

Schemes. IACR Cryptology ePrint Archive, Report 2024/019, 2024. https://eprint.iacr.org/2024/019.

[3] Hao Guo. Cryptanalysis of MPPK/DS. IACR Cryptology ePrint Archive, Report 2023/453, 2023. https://eprint.iacr.org/2023/453.

[4] sergelen02. Test: add HPPK DS security regression suite. Draft GitHub pull request 1, created 9 September 2026. https://github.com/sergelen02/HPPK_2/pull/1. Public reproduction commit 782f51815b011dbd16ac4dd5de48757e0fb69bf4.

[5] D-HPPK KEM preprint. D-HPPK KEM: A Defactorized Homomorphic Polynomial Public Key Encapsulation Mechanism for IND-CCA2 Security. Preprints.org manuscript 202608.1587, version 3, September 2026. https://www.preprints.org/manuscript/202608.1587.

[6] xtaci. hppk public source repository. Commit a66c951eef2f231a228e8121a7a6391b991146e1. https://github.com/xtaci/hppk.